

Capstone Project 1 Report

By: Sahil Bohra

1.) ADLS Gen 2

Step 1 : bronze layer

A screenshot of a Jupyter Notebook interface. The top bar shows a play button, a checkmark, the time '03:35 PM (1m)', and a tab labeled '1'. On the right, there are icons for Python, a trash can, a search icon, and a settings menu, along with the text 'Generate (Ctrl + I)'. The notebook contains the following Python code:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("BronzeIngestionADLS").getOrCreate()

# -----
# 1. Source ADLS path (replace with your container + storage account)
# -----
source_path = "abfss://raw-sahil@sahilstorageaccount81.dfs.core.windows.net/retail_iot_data.csv"

# -----
# 2. Read raw files (CSV + JSON supported)
# -----
df_raw = spark.read.format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
    .load(source_path)

# 3. Write to Bronze Delta Table in Unity Catalog
# -----
df_raw.write.format("delta") \
    .mode("append") \
    .saveAsTable("sahil_catalog_project.sahil_schema_final.sales_bronze")

# -----
# 4. Validate ingestion
# -----
```

Code for Bronze Layer:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("BronzeIngestionADLS").getOrCreate()

# -----
# 1. Source ADLS path (replace with your container + storage account)
# -----
source_path = "abfss://raw-
sahil@sahilstorageaccount81.dfs.core.windows.net/retail_iot_data.csv"

# -----
# 2. Read raw files (CSV + JSON supported)
# -----
df_raw = spark.read.format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
```

```

.load(source_path)

# 3. Write to Bronze Delta Table in Unity Catalog
# -----
df_raw.write.format("delta") \
    .mode("append") \
    .saveAsTable("sahil_catalog_project.sahil_schema_final.sales_bronze")

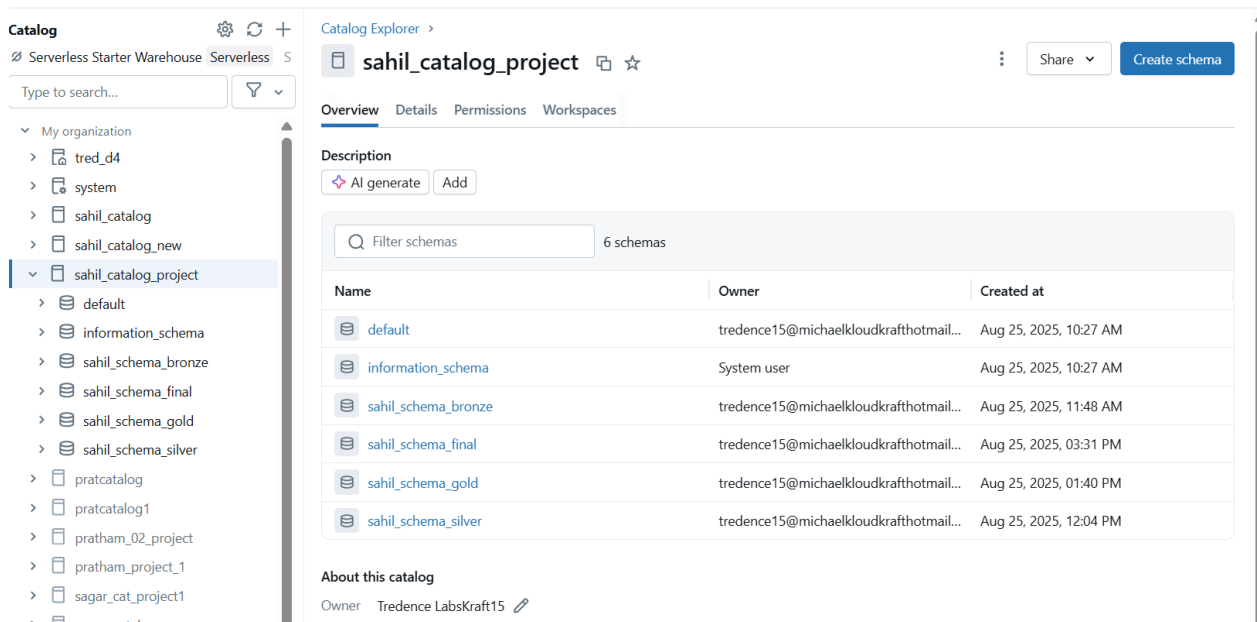
# -----

# 4. Validate ingestion
# -----
display(spark.table("sahil_catalog_project.sahil_schema_final.sales_bronze"))

```

Directly read the csv file from my storage account after uploading the file in my storage account and simultaneously made a bronze layer which contain the raw data.

Step 2 : Silver Layer



The screenshot shows the Unity Catalog Explorer interface. On the left, a sidebar lists various catalogs and schemas. The main panel displays the 'sahil_catalog_project' catalog, which contains six schemas: default, information_schema, sahil_schema_bronze, sahil_schema_final, sahil_schema_gold, and sahil_schema_silver. The 'sahil_schema_silver' schema is highlighted in blue. Below the schema list, there is a table showing the details of the schemas, including their names, owners, and creation times.

Name	Owner	Created at
default	tredence15@michaelkloudkrafthotmail...	Aug 25, 2025, 10:27 AM
information_schema	System user	Aug 25, 2025, 10:27 AM
sahil_schema_bronze	tredence15@michaelkloudkrafthotmail...	Aug 25, 2025, 11:48 AM
sahil_schema_final	tredence15@michaelkloudkrafthotmail...	Aug 25, 2025, 03:31 PM
sahil_schema_gold	tredence15@michaelkloudkrafthotmail...	Aug 25, 2025, 01:40 PM
sahil_schema_silver	tredence15@michaelkloudkrafthotmail...	Aug 25, 2025, 12:04 PM

Below the table, there is a section titled 'About this catalog' which shows the owner as 'Tredence LabsKraft15'.

Created a new catalog and in this I made a schema and store all the 3 layers in it.

03:43 PM (20s) 2 Python

```
.saveAsTable("sahil_catalog_project.sahil_schema_final.sales_silver")

# Display
display(spark.table("sahil_catalog_project.sahil_schema_final.sales_silver"))
```

► (4) Spark Jobs

- df_sales: pyspark.sql.connect.dataframe.DataFrame = [OrderID: string, CustomerID: string ... 12 more fields]
- df_sales_clean: pyspark.sql.connect.dataframe.DataFrame = [event_time: timestamp, store_id: string ... 12 more fields]
- df_sales_cleaned: pyspark.sql.connect.dataframe.DataFrame = [event_time: timestamp, store_id: string ... 12 more fields]
- df_sales_deduped: pyspark.sql.connect.dataframe.DataFrame = [event_time: timestamp, store_id: string ... 12 more fields]
- df_sales_normalized: pyspark.sql.connect.dataframe.DataFrame = [event_time: timestamp, store_id: string ... 12 more fields]

	event_time	store_id	order_id	sales_amount	region	cust_id	
1	2023-01-01T00:20:04.000+00:00	Store-020	96af0d40-093d-4f20-89ed-49b23e469436	751.49	West	CUST-9364	
2	2023-01-01T01:50:21.000+00:00	Store-010	bc92cf60-fe33-443b-82be-fe168c35f2eb	3283.92	South	CUST-4432	
3	2023-01-01T02:51:02.000+00:00	Store-018	d2999758-ee33-4e2e-809d-117bb39c7e07	1951.24	South	CUST-5979	
4	2023-01-01T02:59:19.000+00:00	Store-014	06b209f8-1eaa-4fe4-a810-df490b6743fd	2854.95	North	CUST-2143	
5	2023-01-01T03:51:12.000+00:00	Store-014	fc36bf0f-6bfd-4fd1-9362-6f9163f68931	1103.25	South	CUST-9039	
6	2023-01-01T04:48:16.000+00:00	Store-011	b864e30a-28c9-4e6a-8cc9-75c320efc8cf	2108.1	East	CUST-7112	
7	2023-01-01T06:22:20.000+00:00	Store-020	91f42069-91ca-4b99-801d-cd38085c8aab	4974.48	North	CUST-2373	
8	2023-01-01T06:51:49.000+00:00	Store-015	486599d5-219d-4059-9536-61d984332e42	648.48	East	CUST-9446	

Source Code for silver layer:

```
from pyspark.sql.functions import col, when, row_number, lit, trim
```

```
from pyspark.sql.window import Window
```

```
# Read from Bronze
```

```
df_sales = spark.table("sahil_catalog_project.sahil_schema_final.sales_bronze")
```

```
# Fix AnomalyFlag nulls and cast to string
```

```
df_sales = df_sales.withColumn(
    "AnomalyFlag",
    when(col("AnomalyFlag").isNull(), lit("None"))
    .otherwise(col("AnomalyFlag").cast("string"))
)
```

```
# Standardize column names + include anomaly_type
```

```
df_sales_cleaned = df_sales.select(
    col("OrderTime").cast("timestamp").alias("event_time"),
```

```

trim(col("StoreID")).cast("string").alias("store_id"), # KEEP as string (no nulls from int cast)

col("OrderID").cast("string").alias("order_id"),

col("SalesAmount").cast("double").alias("sales_amount"),

col("Region").cast("string").alias("region"),

col("CustomerID").cast("string").alias("cust_id"),

col("Quantity").cast("int").alias("quantity"),

col("UnitPrice").cast("double").alias("unit_price"),

col("DeviceType").cast("string").alias("device_type"),

col("DeviceStatus").cast("string").alias("device_status"),

col("Product").cast("string").alias("product"),

col("Temperature").cast("double").alias("temperature"),

col("AnomalyFlag").alias("anomaly_flag"),

col("AnomalyType").cast("string").alias("anomaly_type")
).na.drop(subset=["store_id", "order_id", "sales_amount"])

# Deduplicate

window_spec = Window.partitionBy("event_time", "store_id", "order_id").orderBy(col("event_time").desc())

df_sales_deduped = df_sales_cleaned.withColumn("row_num", row_number().over(window_spec)) \
    .filter("row_num = 1").drop("row_num")

# Normalize product

df_sales_normalized = df_sales_deduped.withColumn(
    "product", when(col("product").isNull(), "Unknown").otherwise(col("product"))
)

df_sales_clean = (df_sales_normalized
    .fillna({"anomaly_type": "None"})
)

```

```
# Save to Silver
```

```
df_sales_clean.write.format("delta") \

.mode("overwrite") \

.option("overwriteSchema", "true") \

.saveAsTable("sahil_catalog_project.sahil_schema_final.sales_silver")
```

```
# Display
```

```
display(spark.table("sahil_catalog_project.sahil_schema_final.sales_silver"))
```

Created a silver layer by removing some anomalies and assigned the correct data format for all columns and also normalization of some column is done in silver layer.

Step 3 : Gold Layer



```
gold_daily_enriched.write.format("delta").mode("overwrite") \
.option("overwriteSchema", "true") \
.saveAsTable("sahil_catalog_project.sahil_schema_final.daily_enriched_facts")

# Done ✓
```

▶ (31) Spark Jobs

- ▶ anomaly_trend_store_daily: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, event_date: date ... 4 more fields]
- ▶ by_store_day: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, event_date: date ... 6 more fields]
- ▶ daily_sales_per_region: pyspark.sql.connect.dataframe.DataFrame = [region: string, event_date: date ... 3 more fields]
- ▶ df: pyspark.sql.connect.dataframe.DataFrame = [event_time: timestamp, store_id: string ... 16 more fields]
- ▶ gold_daily_enriched: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, event_date: date ... 9 more fields]
- ▶ monthly_sales_per_region: pyspark.sql.connect.dataframe.DataFrame = [region: string, event_month: string ... 3 more fields]
- ▶ region_monthly_totals: pyspark.sql.connect.dataframe.DataFrame = [region: string, event_month: string ... 3 more fields]
- ▶ region_tiers: pyspark.sql.connect.dataframe.DataFrame = [region: string, event_month: string ... 5 more fields]
- ▶ store_ranked: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_total_sales: double ... 2 more fields]
- ▶ store_tiers: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_total_sales: double ... 3 more fields]
- ▶ store_totals: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_total_sales: double ... 1 more field]
- ▶ weighted_daily: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, event_date: date ... 4 more fields]
- ▶ weighted_monthly: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, event_month: string ... 4 more fields]

/databricks/spark/python/pyspark/sql/connect/expressions.py:1061: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving all data to a single partition, this can cause serious performance degradation.
warnings.warn()

Source code for gold layer:

```
# GOLD LAYER: Aggregation & Enrichment
```

```

from pyspark.sql import functions as F

from pyspark.sql.window import Window


# =====

# 0) Setup

# =====

# spark.sql("CREATE CATALOG IF NOT EXISTS capstone_retail")

# spark.sql("CREATE SCHEMA IF NOT EXISTS capstone_retail.gold")


silver_tbl = "sahil_catalog_project.sahil_schema_final.sales_silver"


df = spark.table(silver_tbl)


# Dates & helpers

df = df.withColumn("event_date", F.to_date("event_time")) \

    .withColumn("event_month", F.date_format("event_time", "yyyy-MM")) \

    .withColumn("is_anomaly", F.when(F.lower(F.col("anomaly_flag")) != "none", F.lit(1)).otherwise(F.lit(0))) \

    .withColumn("w_sales", F.when(F.col("is_anomaly") == 1, F.lit(0.5)).otherwise(F.lit(1.0))) # weight:
anomaly=0.5, normal=1.0


# ===== Optional enrichment: Region =====

# Try to join a store dimension table if you have one:

# capstone_retail.dim.store_dim(store_id STRING, region STRING, ...)

# If it doesn't exist, default region='Unknown'

# has_store_dim = spark.catalog.tableExists("sahil_catalog_project", "dim", "store_dim") \

#         if hasattr(spark.catalog, "tableExists") else False


# if has_store_dim:

#     store_dim = spark.table("sahil_catalog_project.dim.store_dim").select(

#         F.col("store_id").cast("string").alias("store_id_dim"),

```

```

#     F.col("region").cast("string").alias("region")

# )

# df = df.join(store_dim, df.store_id == store_dim.store_id_dim, "left") \
#     .drop("store_id_dim") \
#     .withColumn("region", F.coalesce(F.col("region"), F.lit("Unknown")))
# else:
#     df = df.withColumn("region", F.lit("Unknown"))

# =====

# 1) Daily / Monthly Sales per Region
# =====

daily_sales_per_region = df.groupBy("region", "event_date").agg(
    F.countDistinct("order_id").alias("orders"),
    F.sum("sales_amount").alias("total_sales"),
    F.avg("temperature").alias("avg_temperature")
)

monthly_sales_per_region = df.groupBy("region", "event_month").agg(
    F.countDistinct("order_id").alias("orders"),
    F.sum("sales_amount").alias("total_sales"),
    F.avg("temperature").alias("avg_temperature")
)

daily_sales_per_region.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("sahil_catalog_project.sahil_schema_final.daily_sales_per_region")

monthly_sales_per_region.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \

```

```

.saveAsTable("sahil_catalog_project.sahil_schema_final.monthly_sales_per_region")

# =====

# 2) Device Anomaly Trend per Store (daily)
# =====

anomaly_trend_store_daily = df.groupBy("store_id", "event_date").agg(
    F.count("*").alias("events"),
    F.sum("is_anomaly").alias("anomaly_events"),
    F.sum(F.when(F.col("is_anomaly")==0, 1).otherwise(0)).alias("normal_events")
).withColumn(
    "anomaly_rate", F.col("anomaly_events") / F.col("events")
)

anomaly_trend_store_daily.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("sahil_catalog_project.sahil_schema_final.device_anomaly_trend_per_store")

# =====

# 3) Conversion Rate Impact when devices fail
# (Proxy: compare order counts & sales when anomaly vs not, per store-day)
# =====

by_store_day = df.groupBy("store_id", "event_date").agg(
    F.countDistinct(F.when(F.col("is_anomaly")==1, F.col("order_id"))).alias("orders_anomaly"),
    F.countDistinct(F.when(F.col("is_anomaly")==0, F.col("order_id"))).alias("orders_normal"),
    F.sum(F.when(F.col("is_anomaly")==1, F.col("sales_amount")).otherwise(F.lit(0.0))).alias("sales_anomaly"),
    F.sum(F.when(F.col("is_anomaly")==0, F.col("sales_amount")).otherwise(F.lit(0.0))).alias("sales_normal")
).withColumn(
    "order_impact_pct",
    F.when(F.col("orders_normal") > 0,

```



```

        (F.col("orders_anomaly") - F.col("orders_normal")) / F.col("orders_normal"))
        .otherwise(F.lit(None).cast("double"))
    ).withColumn(
        "sales_impact_pct",
        F.when(F.col("sales_normal") > 0,
            (F.col("sales_anomaly") - F.col("sales_normal")) / F.col("sales_normal"))
        .otherwise(F.lit(None).cast("double"))
    )

by_store_day.write.format("delta").mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("sahil_catalog_project.sahil_schema_final.conversion_impact_per_store_day")

# =====

# 4) Store tiering (High/Medium/Low performers)

# Rule: total sales per store across dataset.

# High = >=80th percentile, Low = <=20th percentile, else Medium.

# =====

store_totals = df.groupBy("store_id").agg(
    F.sum("sales_amount").alias("store_total_sales"),
    F.countDistinct("order_id").alias("store_total_orders")
)

# Percentile ranks over all stores

w_all = Window.orderBy(F.col("store_total_sales"))

store_ranked = store_totals.withColumn("pct_rank", F.percent_rank().over(w_all))

store_tiers = store_ranked.withColumn(
    "store_tier",

```

```

F.when(F.col("pct_rank") >= 0.80, F.lit("High"))

.when(F.col("pct_rank") <= 0.20, F.lit("Low"))

.otherwise(F.lit("Medium"))

)

```

```

store_tiers.write.format("delta").mode("overwrite") \

.option("overwriteSchema", "true") \

.saveAsTable("sahil_catalog_project.sahil_schema_final.store_tiers")

```

```

# =====

```

```

# 5) Weighted average of sales vs anomalies

```

```

# Weighted avg sales per store-day and per store-month,
# with weights: normal=1.0, anomaly=0.5 (set above as w_sales).

```

```

# =====

```

```

weighted_daily = df.groupBy("store_id", "event_date").agg(

    (F.sum(F.col("sales_amount") * F.col("w_sales")) / F.sum("w_sales")).alias("weighted_avg_sales"),

    F.sum("is_anomaly").alias("anomaly_events"),

    F.count("*").alias("events"),

    (F.sum("is_anomaly")/F.count("*")).alias("anomaly_rate")

)

```

```

weighted_monthly = df.groupBy("store_id", "event_month").agg(

    (F.sum(F.col("sales_amount") * F.col("w_sales")) / F.sum("w_sales")).alias("weighted_avg_sales"),

    F.sum("is_anomaly").alias("anomaly_events"),

    F.count("*").alias("events"),

    (F.sum("is_anomaly")/F.count("*")).alias("anomaly_rate")

)

```

```

weighted_daily.write.format("delta").mode("overwrite") \

```

```

.option("overwriteSchema", "true") \

.saveAsTable("sahil_catalog_project.sahil_schema_final.weighted_sales_kpis_daily")


weighted_monthly.write.format("delta").mode("overwrite") \

.option("overwriteSchema", "true") \

.saveAsTable("sahil_catalog_project.sahil_schema_final.weighted_sales_kpis_monthly")


# =====

# (Nice to have) Region-level tiers using monthly totals (optional)

# =====

region_monthly_totals = monthly_sales_per_region.withColumnRenamed("total_sales",
"region_monthly_sales")


w_region = Window.partitionBy("region").orderBy("region_monthly_sales")

region_tiers = region_monthly_totals.withColumn("pct_rank", F.percent_rank().over(w_region)) \

.withColumn(

    "region_tier",

    F.when(F.col("pct_rank") >= 0.80, F.lit("High"))

    .when(F.col("pct_rank") <= 0.20, F.lit("Low"))

    .otherwise(F.lit("Medium"))

)


region_tiers.write.format("delta").mode("overwrite") \

.option("overwriteSchema", "true") \

.saveAsTable("sahil_catalog_project.sahil_schema_final.region_tiers_monthly")

```

Created a gold layer in which I made all the aggregation required in the question.

Step 4 : Connected to Power BI for Visualization

The screenshot displays the Databricks SQL interface for configuring a 'Serverless Starter Warehouse'. The left sidebar lists various navigation options, including 'SQL Warehouses'. The main panel shows the 'Connection details' tab, which provides the necessary information to connect to the warehouse. The fields include:

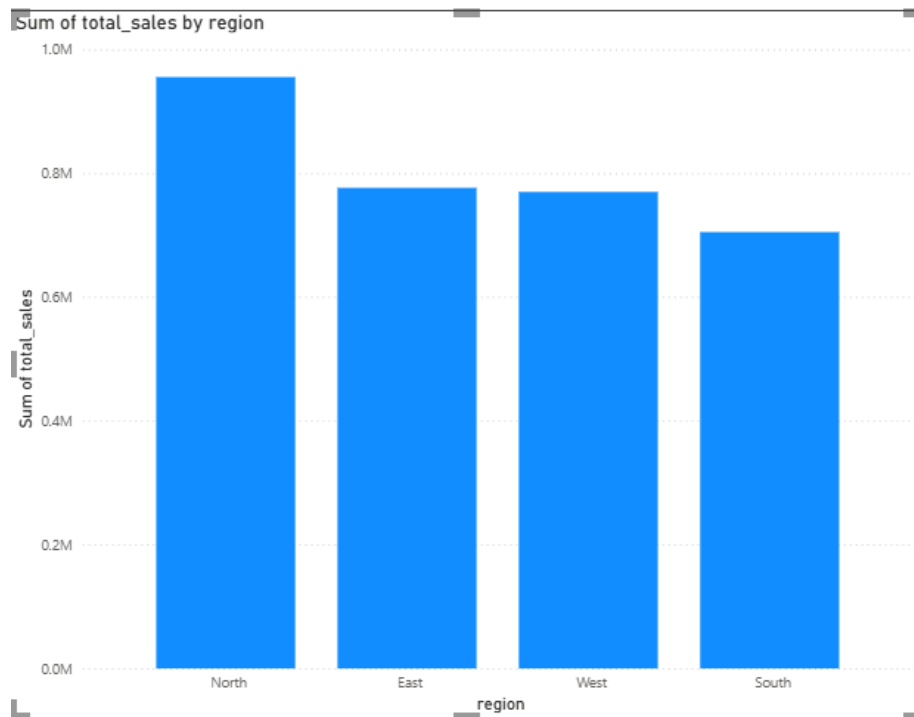
- Server hostname:** adb-3325880049345578.18.azuredatabricks.net
- HTTP path:** /sql/1.0/warehouses/ecb311e1d1ddb7c0
- JDBC URL:** 2.6.25 or later
- OAuth URL:** https://adb-3325880049345578.18.azuredatabricks.net/oidc

Below these fields, there is a row of icons for various tools that can connect to the warehouse: Tableau, Power BI, dbt, Python, Java, Node.js, Go, and a 'More tools' button.

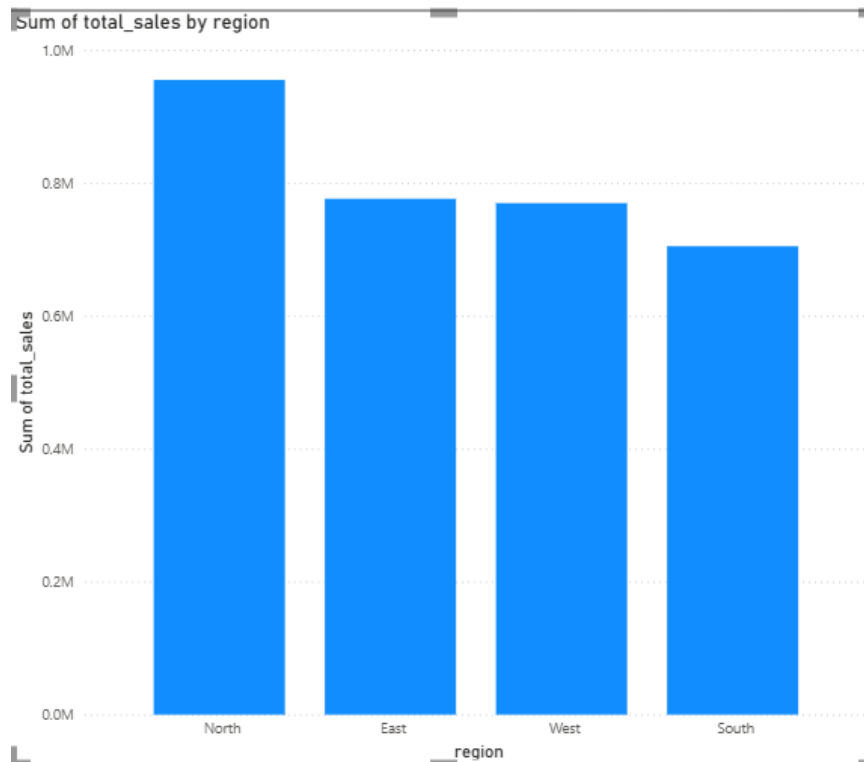
connect it with power bi to generate the report.

Step 5 : Power BI Results

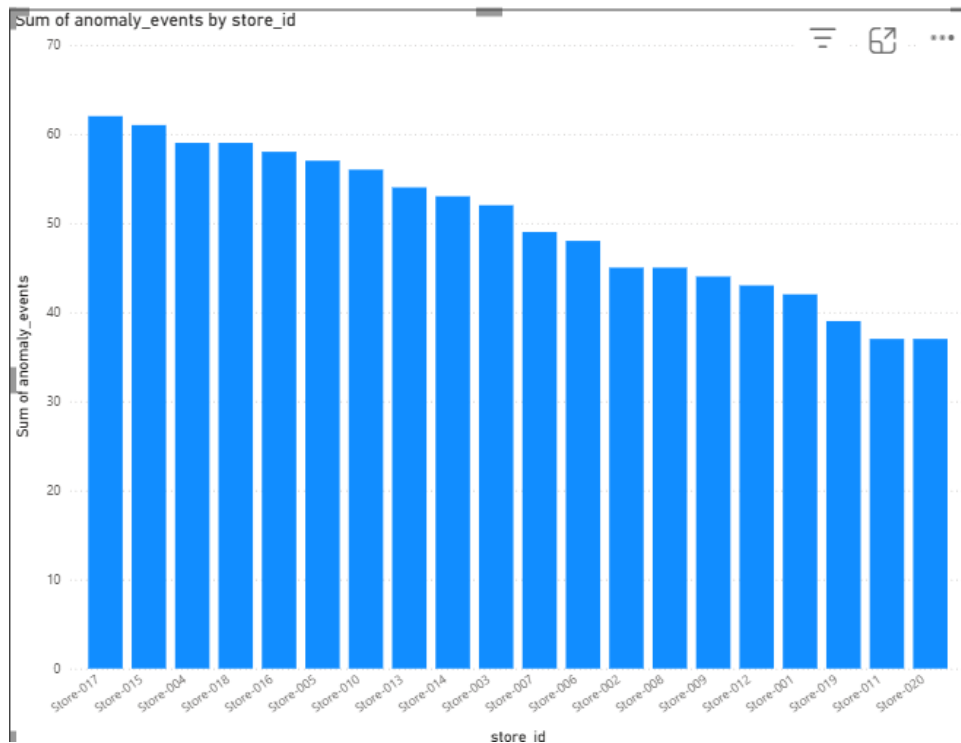
i) Avg. Daily Sales per Region



ii) Avg. Monthly Sales per region



iii) Device Anomaly Trend per Store



2) Event Hub

Step 1 : Created a event workspace on azure and event hub in it .

sahil-workspace Event Hubs Namespace

Search

Event Hub Delete Refresh Give feedback

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Data Explorer
- Resource visualizer
- Events
- Settings
- Entities
- Monitoring
- Automation
- Help

You can start generating test data or inspect data that has already been sent with the new Azure Event Hubs Data Explorer. Click on this message to try the feature!

Essentials [JSON View](#)

Resource group (move) LabsKraft2027	Created Monday, August 25, 2025 at 15:39:49 GMT+5:30
Status Succeeded	Updated Monday, August 25, 2025 at 15:40:05 GMT+5:30
Location West US 2	Zone Redundancy Enabled
Subscription (move) LabsKraft	Pricing tier Basic
Subscription ID d008c7cc-9795-4292-bf79-74ae52cda63d	Throughput Units 1 unit
Host name sahil-workspace.servicebus.windows.net	Auto-inflate throughput units Not Supported
	Local Authentication Enabled

Tags [\(edit\)](#)
[Add tags](#)

Step 2 : Created a VM to send data to event hub by connecting it with.

The screenshot shows the Azure portal interface for a virtual machine named 'sahil-VM'. The left sidebar contains a navigation menu with options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Resource visualizer, Connect, Networking, Settings, Availability + scale, Security, Backup + disaster recovery, Operations, and Monitoring. The main area displays the 'Essentials' tab for the VM, showing details such as Resource group (LabsKraft2027), Status (Running), Location (South India), Subscription (LabsKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), Operating system (Linux (ubuntu 24.04)), Size (Standard B1s (1 vcpu, 1 GiB memory)), Public IP address (4.247.31.230), Virtual network/subnet (rg-musharaf-capstone-vnet/default), DNS name (Not configured), Health state (-), and Time created (8/25/2025, 11:00 AM UTC). A 'JSON View' link is visible in the top right corner of the Essentials section.

[sahilVM_81@4.247.31.230](#) : connected to VM by using this command in cmd.

Step 3: Generate the data using the below python code and run the script in cmd to send the data to event hub.

```
import random
import time
import uuid
from datetime import datetime
from azure.eventhub import EventHubProducerClient, EventData

# -----
# CONFIGURATION
# -----
EVENT_HUB_CONNECTION_STR = "Endpoint=sb://sahil-
workspace.servicebus.windows.net/;SharedAccessKeyName=sahil_sas_key;SharedAccess
sKey=+MD7H90F6wFcFN87G6P2YQlykZLVJJRES+AEhA+6RDU=;EntityPath=sahil-event-
hub"
EVENT_HUB_NAME = "sahil-event-hub"
```

```

# -----
# Function to generate one IoT row
# -----
def generate_row():
    quantity = random.randint(1, 10)
    unit_price = round(random.uniform(50, 500), 2)
    sales_amount = round(quantity * unit_price, 2)
    device_status = random.choice(["Active", "Inactive", "Error"])
    anomaly_flag = 1 if device_status != "Active" else 0
    anomaly_type = random.choice(["Overheat", "LowBattery", "Disconnected"]) if
anomaly_flag else "None"

    return {
        "OrderID": str(uuid.uuid4()),
        "CustomerID": f"CUST{random.randint(1000,9999)}",
        "Region": random.choice(["North","South","East","West"]),
        "StoreID": random.choice(["Store001","Store002","Store003"]),
        "Product": random.choice(["Laptop","Phone","Monitor","Keyboard","Mouse"]),
        "Quantity": quantity,
        "UnitPrice": unit_price,
        "SalesAmount": sales_amount,
        "OrderTime": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        "DeviceType": random.choice(["SensorA","SensorB","SensorC"]),
        "Temperature": round(random.uniform(20, 100), 2),
        "DeviceStatus": device_status,
        "AnomalyFlag": anomaly_flag,
        "AnomalyType": anomaly_type
    }

# -----
# Event Hub Producer
# -----
producer = EventHubProducerClient.from_connection_string(
    conn_str=EVENT_HUB_CONNECTION_STR,
    eventhub_name=EVENT_HUB_NAME
)

```



```

# -----
# Send data every second
# -----
try:
    for _ in range(1000): # Send exactly 1000 rows
        row = generate_row()
        event_data = EventData(str(row)) # convert dict to string
        with producer:
            producer.send_batch([event_data])
        print(f"Sent row: {row}")
        time.sleep(1) # 1-second interval
except KeyboardInterrupt:
    print("Streaming stopped by user.")
finally:
    producer.close()

```

Step 4 : Setup event hub configs in databricks

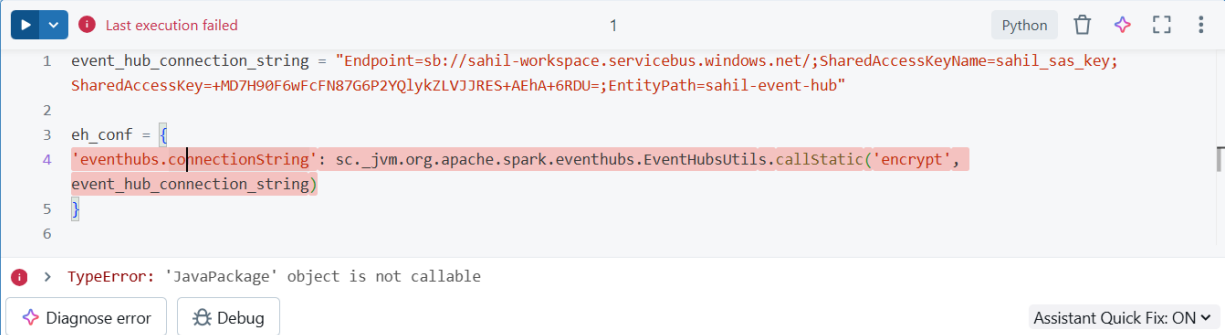
Code:

```

event_hub_connection_string = "" eh_conf = { 'eventhubs.connectionString':
sc._jvm.org.apache.spark.eventhubs.EventHubsUtils.encrypt(event_hub_
connection_string) }

```

It shows the below error :



```

1 event_hub_connection_string = "Endpoint=sb://sahil-workspace.servicebus.windows.net/;SharedAccessKeyName=sahil_sas_key;
  SharedAccessKey=+MD7H90F6wFcFN87G6P2YQlykZLVJJRES+AEhA+6RDU=;EntityPath=sahil-event-hub"
2
3 eh_conf = {
4   'eventhubs.connectionString': sc._jvm.org.apache.spark.eventhubs.EventHubsUtils.callStatic('encrypt',
  event_hub_connection_string)
5 }
6
> TypeError: 'JavaPackage' object is not callable

```

We don't have the admin access to add the required library to the compute.